CELESTIAL MECHANICS × AI · 2026

From the next token to the next theorem



Bowen Liu

Bowen Liu 刘博文

Periodic N-body solutions, elliptic relative equilibria and linear stability.

2013–2018 **Nankai University** · Chern Institute · Math PhD

2018–2020 **University of Augsburg** · Postdoc

2020–2022 **Shanghai Jiao Tong** · Wu Wen-Tsun Asst. Prof.

2022–2024 **Huawei** · Principal Engineer

2024–now **US e-commerce** · Applied Researcher

CELESTIAL MECHANICS RESEARCH

Periodic N-body: existence & stability

Restricted four-body elliptic equilibria

Linear instability of elliptic rhombi

Homographic Lennard–Jones solutions

> Symplectic indices · Variational methods

Overview: from token to theorem

Click a section card to jump; click a tag for a key page.

00 · OPENING + TOKEN

Token

Ten integers · vocabulary · billing

p. 4–9

About me | Ten integers | Vocabulary

01 · MODEL

One function

Distribution

p. 10

Distribution

02 · TRAINING & PROBLEM SOLVING

Learning from reward

RL loop · math and code · GRPO

p. 11–23

RL loop | GRPO formula | Stateless

03 · SYSTEMS & ENGINEERING

Put the model in a system

Context · harness · skills · long runs

p. 24–31

Harness | System map | Skill | Long runs

04 · AI × MATH

From tools to systems

Acceleration · mechanisms · theorem cracking

p. 32–43

DAG/MCTS | 722 papers | NS | System

05 · CONCLUSION

From token to theorem

Probability · verification · direction

p. 44

Close

What the model sees: ten integers

EN · ONE SENTENCE

Celestial mechanics is the poetry of orbits.

Cel	estial	_mechanics	_is	_the	_poetry	_of	_or	bits	.
69126	90010	52347	382	290	38153	328	503	24043	13

46 characters → **10** tokens → 10 integers · _ marks a leading space

CHINESE · FOUNDATIONS OF CELESTIAL MECHANICS

本书系统地介绍了天体力学

本	书	系统	地	介绍	了	天	体	力	学
4087	20121	31936	5557	37887	4531	867	7074	11343	8590

12 characters → **10** tokens → 10 integers · two-character words often map to one token

01

Where do these integers come from?

02

How does it predict the next one?

03

How do we put it to work?

Tokenization is table lookup

```
Three-body problem is a classic problem in celestial mechanics.
```

Three	-body	problem	_is	_a	_classic	_problem	_in	_celestial	_mechanics	.
25843	19146	4792	382	261	13686	4792	306	134437	52347	13

_ marks a leading space · Celestial → Cel + estial

≈5
characters / English word
including the space, typical

≈4
characters / token
English rule of thumb

≈1.3
tokens / English word
≈ 5 ÷ 4

50k–200k
typical vocabulary size
50k – 200k

WHY Characters: little information, 4–5× longer → Words: vocabulary explodes (millions) → **Subwords: the compromise**

> Subwords are an engineering compromise between **sequence length** and **vocabulary size**.

The vocabulary: a fixed lookup table

ID	TOKEN	
290	_the	
2392	_book	
2500	This	
18864	_fundamental	
23753	_concepts	
52347	_mechanics	
134437	_celestial	← query
199999	<|endoftext|>	special

8 rows shown — 200,019 rows in total

200,019

o200k · GPT-4o generation

- **0**
- **50k** GPT-2
- **100k** GPT-4 gen
- **128k** Llama 3
- **200k** GPT-4o gen

200,019 × 4,096 ≈ **820M parameters**

> An ID is a **row number**; changing models means changing the table.

Tokens: the model's unit and the billing unit



≈ 0.3 token

per English character

≈ 0.6 token

per Chinese character

> Token is both the smallest unit of the model and the smallest unit of the bill.

Where the money goes on a single call

价格[2]	百万tokens输入（缓存命中）	空闲时段	0.02元
		高峰时段	0.04元
	百万tokens输入（缓存未命中）	空闲时段	1元
		高峰时段	2元
	百万tokens输出	空闲时段	4元
		高峰时段	8元

Input 100k + output 2k

¥0.01 cache hit

¥0.11 cache miss

> The same context, cached or not, differs by an **order of magnitude**.

Text becomes an ID, then a vector

PIECE

`problem`

A unit after tokenization

→

ID

4792

Row 4792 of the vocabulary

→

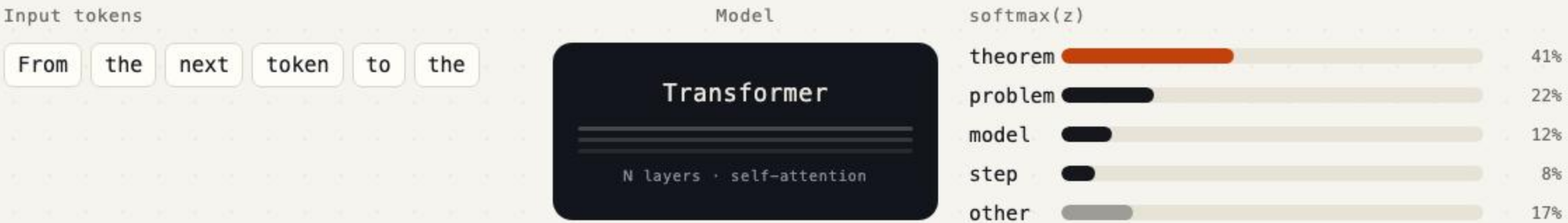
VECTOR

0.12 −0.44 0.08 …

4,096 dimensions · learned

> The ID is an **address**; the vector is the **content**.

What the model does: input → probability distribution



$$z = f_\theta\left(x_{1:t}\right) \in \mathbb{R}^{|V|} \qquad p_\theta\left(y \mid x_{1:t}\right) = \frac{\exp\left(z_y\right)}{\sum_{v \in V} \exp\left(z_v\right)}$$

$|V| = 200{,}019$

$\Sigma_y\, p_\theta(y \mid x_{1:t}) = 1$

> A scorer: it gives every token in the vocabulary a **score**, then turns scores into **probabilities**.

Training: pretraining and post-training

STAGE 01

Pretraining

Large-scale text, code and other data

Learn language, knowledge and patterns

→

STAGE 02

Post-training

SFT · SUPERVISED FINE-TUNING

Learn by example

Shape responses and task behavior.

RL · OUR FOCUS

Learn from reward

Try, evaluate, improve the policy.

> Our focus is **RL**: improving **problem-solving** through reward.

RL: actions, outcomes and feedback

Act in an environment; use outcome feedback to improve action choices.

Observe environment → Take action → Outcome → Feedback / reward → Improve policy ↺

ENVIRONMENT

Produce an outcome

Actions affect the environment; new observations return.

FEEDBACK

Evaluate the result

Reward expresses progress or success.

POLICY

Improve action choices

Use experience to seek higher long-run return.

> RL uses **outcome feedback** to learn **how to choose actions**.

Games made reinforcement learning visible

2015

Atari · DQN

Pixels → actions

Reward: game score

2016

AlphaGo

Self-play + search

Reward: winning

2019

OpenAI Five

Dota 2 team play

Reward: match outcome

2019

AlphaStar

StarCraft II strategy

Reward: match outcome

- Explicit rules
- Measurable outcomes
- Many repeatable trials

> The game engine supplies an **environment and feedback**.

A score can teach a strategy

Breakout: move the paddle, hit the ball, clear the bricks.

TRY

Different moves

Explore timing and direction.

OBSERVE

Score the outcome

Miss the ball, or clear bricks.

LEARN

Better strategies

Learn actions with delayed benefits.

`Repeated trials + outcome feedback → a policy that seeks higher reward.`

A useful environment lets the agent **try, evaluate and try again**.

The GRPO objective

DeepSeekMath: group-relative advantage with clipping and a KL constraint.

$$J_{\mathrm{GRPO}}\left(\theta\right) = E_{q \sim P(Q),\, \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\mathrm{old}}}} \left[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} L_{i,t} \right]$$

PER-TOKEN TERM

$$L_{i,t} = \min\left[\rho_{i,t}\left(\theta\right) \hat{A}_{i,t}, \mathrm{clip}\left(\rho_{i,t}\left(\theta\right), 1-\varepsilon, 1+\varepsilon \right) \hat{A}_{i,t} \right] - \beta D_{\mathrm{KL}}\left[\pi_\theta \middle\| \pi_{\mathrm{ref}} \right]$$

PROBABILITY RATIO

$$\rho_{i,t}\left(\theta\right) = \frac{\pi_\theta\left(o_{i,t} \mid q, o_{i,<t}\right)}{\pi_{\theta_{\mathrm{old}}}\left(o_{i,t} \mid q, o_{i,<t}\right)}$$

GROUP-RELATIVE ADVANTAGE

$$\hat{A}_{i,t} = \frac{r_i - \mathrm{mean}\left(r_1, \ldots, r_G\right)}{\mathrm{std}\left(r_1, \ldots, r_G\right)}$$

> Raise high-advantage responses while **clip and KL** limit policy drift.

GRPO: compare a group of answers

No critic; use relative performance within a group.

One question → Sample a group → Score each → Relative advantage ↺

ONE QUESTION · G = 4 ANSWERS

Reward / group advantage / update

A · r = 0.20	−1.5 ↓
B · r = 0.90	**+1.1 ↑**
C · r = 0.50	−0.4 ↓
D · r = 0.80	**+0.7 ↑**

COMPETITION WITHIN A GROUP

Compare answers to one question, not absolute scores.

ADVANTAGE SETS THE DIRECTION

- **$\hat{A} > 0$** Raise this answer's probability
- **$\hat{A} < 0$** Lower this answer's probability
- **$\hat{A} \approx 0$** No relative edge; little gradient

NO CRITIC

Group mean is the baseline; no value model.

> Within one question, let **better answers win**.

Math and code share a feedback advantage

01 · CLOSED ENVIRONMENT

Axioms · Runtime

Math uses definitions, axioms and inference rules; code uses a language spec and runtime. The system is self-contained.

02 · TRACEABLE ERRORS

Locate the step

Proofs check inference step by step; tests, traces and line numbers locate code errors.

03 · CLEAN DATA

Parse and filter

Formal objects and code are structured, enabling bulk generation and verification.

Two levers: better policies + more compute for candidate generation and verification.

Closed environment + localizable errors + clean data → **cheap feedback, large-scale trial and error**.

Code and math exercise shared reasoning

CODE PRACTICE

- Track variables and state
- Respect types and constraints
- Decompose and debug

MATH PRACTICE

- State assumptions explicitly
- Maintain symbolic consistency
- Build and check multi-step arguments

Reported transfer: math RL → coding scores · code training → state tracking.

Shared skills support **transfer**; the data mixture determines the gain.

Coding gives an agent a place to act

`Read files` → `Edit` → `Run tests` → `Inspect & revise` ↺

WORKSPACE

Files and specifications define the task.

TOOLS

Changes can be executed immediately.

FEEDBACK

Failures guide the next edit.

> Code connects **generation, action and feedback** in one workspace.

Four tasks, different feedback conditions

Code, math, product design, trading: how do goals, environments and feedback fit?

01 · CODE

Executable checks

- Target: specified behavior
- Feedback: compiler, tests, errors
- Trials: run and revise locally

02 · MATH

Explicit assumptions & goals

- Target: solve or prove under assumptions
- Feedback: answers, proof checks, counterexamples
- Trials: generate and verify candidates

03 · PRODUCT DESIGN

Long real-world feedback loops

- Target: user value and differentiation
- Feedback: slower, plural, empirical
- Judgment: distinctive, sometimes non-consensus

04 · TRADING

Clear reward, changing markets

- Target: profit, risk and cost
- Environment: incomplete info, shifting markets
- Validation: backtests versus future returns

> Checkable goals turn more compute into **systematic search and candidate filtering**.

Correctness is a floor; quality is an ambition

	CODE	MATHEMATICS
CHECKABLE STRUCTURE	Syntax, types, tests, specifications	**Definitions, assumptions, proof steps**
CORRECTNESS	Specified behavior is satisfied	**A claim follows from its assumptions**
QUALITY	Clarity, maintainability, efficiency	**Elegance, generality, usefulness**

> Both communities pursue **good work** beyond a pass/fail result.

Summary: environment, feedback, updates

Well-defined tasks let a model predict, receive checks and iterate in an environment.

Problem + environment → Next-token prediction → Check + trace → Define loss → Backpropagate ↺

01 · ENVIRONMENT

Clear rules & goal

The model can act, try and produce outputs.

02 · FEEDBACK

Evaluate & trace

Check outcomes and locate failing steps.

03 · UPDATE

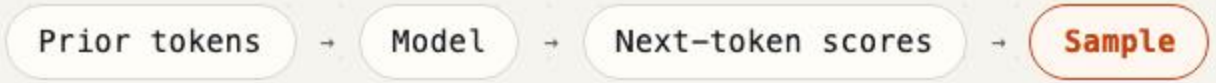
Loss-driven iteration

Backpropagate, update parameters, improve.

> The environment supplies feedback; **loss turns it into parameter updates**.

The model only predicts the next token

Memory, computation, tools and plans are built outside that one action.



One action: score the next token, then sample one.

01 · NO MEMORY OF THE LAST TURN

The client resends the history

02 · NO REAL COMPUTATION

Arithmetic is recall, not computation

03 · NO DIRECT ACTION

Tools execute outside the model

04 · NO PERSISTENT GOAL

Plans are generated text too

> The model supplies one token; **the system supplies the rest**.

After training: four limits, three system capabilities

The model predicts; state, computation and reliability live outside it.

01 · MEMORY & CONTEXT

No persistent state, limited window

- Parameters fixed at inference; knowledge in weights
- A “conversation” resends history
- 4k → 1M, but long inputs still lose the middle

02 · COMPUTATION IS RECALL

Recall and pattern completion

- Arithmetic comes from training patterns, not an algorithm
- Times table: recall, not algorithm
- Errors compound over long tasks

03 · PROBABILISTIC OUTPUT

Same input, different output

- Output is sampled, not a deterministic function
- Errors can sound confident and plausible
- Long runs need stopping and rollback

> Four limits, three system capabilities: **state, computation, reliability**.

The inflation of the context window



COST

Billed per token; long prefills are slower

LATENCY

Prefill and attention costs rise

DILUTION

Middle content gets ignored; signal drowns in noise

> A longer window raises the ceiling; relevance decides what lands.

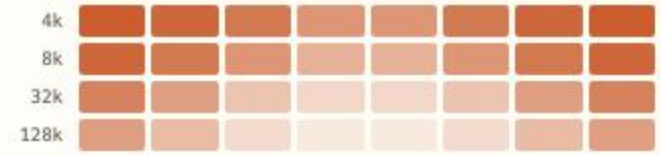
Context and memory: a finite window, used well

The window is a budget, not a warehouse: place the right things, then decide what stays.

01 · FINITE WINDOW

4k → 1M, still lossy

Needle in a haystack: the middle is diluted first.



Capacity ≠ effective use.

02 · PLACEMENT

Workbench, not warehouse

- system · rules
- history · recent turns
- retrieval · relevant docs
- tools · returned results
- current · this question

Too long→retrieve; too scattered→structure; overflow→summarize or sub-agents.

03 · RETENTION

Signal, not logs

Signal
tasks · constraints · decisions · facts

Noise
chat · repetition · stale · raw logs

WRITE
decisions and facts only

RETRIEVE
for this task

COMPRESS
summarize, merge

FORGET
delete stale

> Capacity ≠ effective use; Context decides **what goes in**, Memory decides **what stays**.

Tool calling: let a program do the arithmetic

tool schema

```
{
  "name": "run_python",
  "args": {"code": "str"},
  "returns": "str",
  "limits": ["sandbox", "timeout"]
}
```

one call

```
call  run_python("print(2 * 4**1.5)")
obs   "16.0"
```

```
// T = 2 × 4**1.5 = 16
```

01 DEFINE

Name, arguments, return, boundary; the description is context too.

02 CALL

The model emits a tool_call; it proposes, does not execute.

03 EXECUTE

The runtime validates arguments and runs in a sandbox; timeouts and permissions are system concerns.

04 OBSERVE

stdout / stderr return to the context; the model decides to continue or stop.

> Tool output is still **untrusted input**: validate arguments, limit permissions, set timeouts, keep logs.

> The model proposes, the program executes, **the system owns the boundary**.

Harness: turn probabilistic output into a runnable system

The model proposes; the harness makes the proposal legal, executable, checkable, and stoppable.

A controlled loop

Propose → Validate → Execute → Observe → Verify → Decide

ENGINEERING BOUNDARY

The model only proposes token distributions and tool calls; the security and determinism of the environment are enforced by the **Harness**.

> It does not add intelligence; it makes errors **visible, stoppable, reproducible**.

01 TOOL REGISTRATION & VALIDATION
Check schemas and types; block illegal proposals immediately

02 SANDBOX & PERMISSIONS
What may run; read/write isolation, constrain before execution

03 TIMEOUTS & RETRIES
Prevent infinite loops; enforce timeout limits and retry caps

04 RESULT VERIFICATION
Check correctness via independent assertions; never trust self-claims

05 ROLLBACK & STOPPING
Undo bad changes via snapshots; force-stop on error thresholds

06 LOGS & REPLAY
Record every call and response; ensure full trajectory auditability

> The model provides probability; the harness provides the **boundary**.

Put the model back in the system



01	**LLM**	A probability generator: tokens in, a distribution over the next token out
02	**Prompt**	This call's conditions: system / user / few-shot
03	**Context**	Every token visible to this call; the window is finite
04	**Memory**	State beyond the window: write / retrieve / compress / forget
05	**Agent loop**	Observe → act → observe; state lives outside
06	**Harness**	Tools, sandbox, permissions, verification, logs

> Memory owns state, tool calls compute, harness verifies: together they form an **agent system**.

Skill: package a procedure into a reusable capability

Not a bigger model; a load-on-demand bundle of steps, scripts, and references.

```
skill package
skill/
  SKILL.md        when · steps · checks
  scripts/        deterministic ops
  references/     domain docs
  assets/         templates
```

Outline → Generate → Screenshot → Audit → Export

01 TRIGGER
Name and description: when to use it, when not to

02 STEPS
SKILL.md: what comes first, what follows, where the checks are

03 TOOLS
scripts/: freeze deterministic operations; stop rewriting them

04 ASSETS
references/ and assets/: domain knowledge, templates, examples

A skill is loaded on demand, versioned, and reusable; context budget and tool permissions still apply.

Turn a one-off prompt into a **versioned procedure**.

Long-running: turn the loop into a skill

The loop produces experience; the skill turns it into a procedure the next run can load.

01 · RUN

Loop

`Observe` → `Act` ↺

State lives outside; a run can last hours or days.

02 · DISTILL

Trajectory → steps

`Success` `Failure` `Repair` `Checkpoint`

Drop the accidental; keep what repeats.

03 · CODIFY

Skill

```
SKILL.md
scripts/
references/
```

Load on demand; use it on the next run.

`Loop is runtime; skill is compiled experience; memory is state; harness is the boundary.`

> Long runs do not only finish tasks; they turn practice into an **asset**.

Acceleration: from basic use to gold medals

2023 · BEFORE

Basic use

- Explain concepts, search the literature, write code
- Do simple calculations, often wrong
- Complex proofs still need people
- Role: assistant

→ 2024–26

2025–26 · AFTER

Research level

- IMO 2025 gold level 35/42
- Lean proofs, machine-checkable
- OpenAI 2026: ≈3 Pro-hours of compute per result*
- Role: candidate generator

2024.07 **28/42** silver → 2025.07 **35/42** gold level — 12 months, +7 points

> This is a staircase with **three steps**.

Timeline: three steps

2023–24

Tools: search + verification

2023.12
FunSearch
LLM + evaluator + evolutionary search; a new cap-set construction

2024.01
AlphaGeometry
Neuro-symbolic; IMO geometry 25/30

2024.07
AlphaProof
Lean + RL; IMO 2024 silver 28/42 (+AG2)

2024–25

Reasoning: RL + test-time compute

2024.09
OpenAI o1
RL teaches thinking; test-time compute

2025.01
DeepSeek-R1
RLVR; reasoning behavior emerges; open weights

2025.05
AlphaEvolve
Evolutionary coding agent; 4×4 complex matrix product in 48 multiplications

2025–26

Scale: from benchmarks to open problems

2025.07
IMO 2025
Gold level 35/42; natural language, no tools

2025
Formal gold (claimed)
Aristotle et al.; Lean-checkable

2026.05–10
OpenAI
Unit-distance counterexample; NS / Riemann-related (claimed); 372 result families at launch

> Every leap combines a **generator, a verifier and search**.

How they describe it

> Public material covers part of the method; the full training recipe stays private. Note the asymmetry: some results are easy to verify, hard to prove. Quotes below are **public statements**, excerpted or paraphrased.

DATE	SYSTEM / WORK	PUBLIC STATEMENT	READING
2023.12	**FunSearch**	"pairing a pre-trained LLM ... with an automated evaluator"	search
2024.01	**AlphaGeometry**	"100 million synthetic theorems and proofs"	synthetic data
2024.07	**AlphaProof**	autoformalization → Lean; RL + test-time RL	formalization
2024.09	**o1**	"spend more time thinking through problems"	reasoning
2025.01	**R1**	"directly apply RL ... rule-based rewards"	RLVR
2025.05	**AlphaEvolve**	"evolutionary coding agent"	evolve code
2025.07	**IMO 2025**	"natural language ... no tools"	end to end
2026.09–10	**OpenAI**	NS breakdown claim; 372 families, 722 manuscripts at launch	open problems

> The keywords change; the underlying move stays: **propose, verify, search again**.

The common pattern: generate → verify → search

GENERATE

LLM

Candidate proofs, programs, constructions

→

VERIFY

Lean / tests / numerics

Returns right or wrong, a score, a counterexample

→

SEARCH / LEARN

best-of-N · trees · evolution · RL

Turns the signal into the next policy

↺ Back to the generator: the next round is stronger

MANUAL ONLY

Review stays manual; scale stalls

ONE SHOT

One draw per attempt; luck decides

BOTH

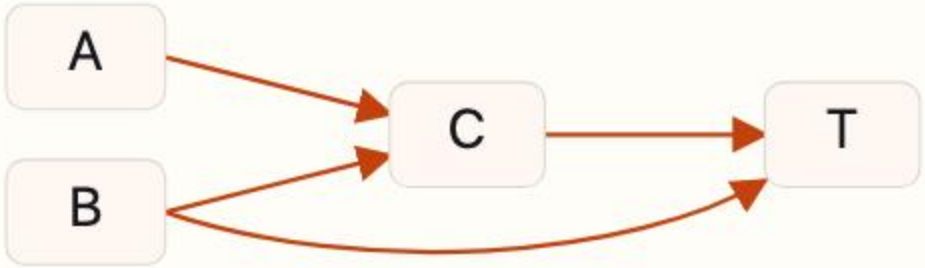
Probability is filtered into correctness, then written back into the model

> The model supplies probability; **the verifier supplies the gradient**.

DAG: dependencies. MCTS: where to search next.

DAG · DIRECTED ACYCLIC GRAPH

A dependency structure



Nodes: lemmas and subgoals

Directed edges: premises → conclusion

No cycles; shared lemmas can be reused

MCTS · MONTE CARLO TREE SEARCH

A search policy



Nodes: states; branches: candidate actions

Select → expand → evaluate → back up

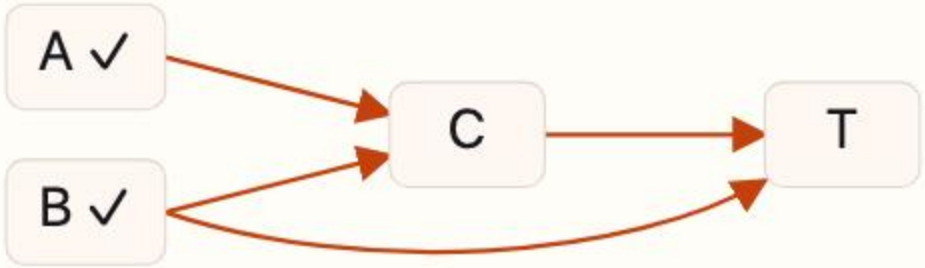
Balance exploration and exploitation

DAG: **"What depends on what?"** MCTS: **"What should we try next?"**

My hypothesis: agents + Lean evolve a proof graph

Keep an external graph / tree of goals, candidate paths, verified lemmas and feedback.

01 · MAINTAIN GOALS AND DEPENDENCIES



Nodes store goals; edges track dependencies

Verified lemmas become reusable nodes

02 · PROPOSE, CHECK AND UPDATE



Select node → agent attempt → Lean check

Keep verified work; revise or switch paths

The graph stores state; agents extend it; Lean checks proofs.

My assessment: real progress, a route still maturing

OpenAI reports ≈4,000 attempted problems; the release covers a subset of that effort.

01 · SYSTEM BOTTLENECKS

Organizing the search

Goal decomposition; node selection

Branch coordination; shared state

Budget allocation; verifier feedback

Stability at 10k-agent scale

02 · MODEL BOTTLENECKS

Finding the key step

Useful lemmas and new constructions

Difficult long-horizon reasoning

Turning ideas into rigorous proofs

> Lean checks formal logic; **system and model jointly set the frontier**.

OpenAI: research mathematics at batch scale

From a single demonstration to a collection for review and follow-on research.

722
Manuscripts at launch

372
Related result families

≈4,000
Problems attempted

WHAT WAS RELEASED

Manuscripts and proof artifacts
Lean formalizations for some results
10 reasoning summaries; compute data

HOW I READ THE NUMBERS

One family: several papers
3 Pro hours: compute equivalent
Truth, novelty, significance: review

> Candidate results arrive in batches; **verification and understanding matter**.

Navier–Stokes: a long-running multiagent effort

OpenAI reports finite-time breakdown in smooth, forced 3D incompressible flow.

$$\partial_t u + (u \cdot \nabla) u = -\nabla p + \nu \Delta u + f, \quad \nabla \cdot u = 0$$

SCOPE OF THE REPORTED RESULT

Positive viscosity; smooth data & force

Whole space $\mathbb{R}^3$ and periodic flow

Finite-time breakdown; Clay C / D

A breakdown construction; unforced NS is separate.

≈10^4	88 h	+17 h	130B
Agents	Find argument	Formalize	Output tokens

Explore in groups → consolidate findings

Share → search further → Lean checks

10k agents → scheduling, checkpoints, retries, stopping

> Check the logic; **also audit the problem–statement–paper correspondence**.

Theorem-cracking system: LLM as CPU

Not just a mathematics problem; a machine for throughput and stability.

WORKLOAD	**Theorem · proof graph**	what to solve
CPU	**LLM**	propose next
SCHEDULER	**Loop + Harness**	concurrency · retries · checkpoints · stop
VERIFIER	**Lean / tools**	correctness · compute
STATE	**Context + Memory**	graph · logs · budget

01 · NOT ONLY MATHEMATICS

Decomposition, scheduling, state, verification

02 · THROUGHPUT

Concurrency, caching, reclamation, cost

03 · STABILITY

Long runs, recovery, auditability

04 · SCALABLE

Swap problems, not the system

> The model is the CPU; **theorem cracking needs the whole machine**.

Failure probability: long chains amplify 1%

The proof graph is read and written constantly; errors compound along dependencies.

END-TO-END SUCCESS · ILLUSTRATIVE

Per step	100	1,000	10,000
99%	37%	0.0043%	≈0
99.9%	90%	37%	0.0045%
99% + 3 retries	≈100%	99.9%	99%

Assumes independent failures; real errors correlate, and retries are not free.

DETECT ERRORS

Lean, tests, assertions first

RETRY SAFELY

Backoff, caps, no duplicate effects

ROLL BACK STATE

Checkpoints, idempotency, recovery

> The system does not prevent every error; **it makes errors non-fatal**.

Systems engineering: six interfaces

Each layer can work alone; the whole machine still may not be reliable.

01 · STATE CONSISTENCY
Concurrent graph edits
Versions, locks, conflict merge

02 · SCHEDULING & BUDGET
Who runs, how long
Concurrency, preemption, cost

03 · FAILURE & RETRY
No duplicate effects
Idempotency, backoff, caps

04 · VERIFICATION INTERFACE
Natural language ↔ Lean
Semantic gaps and assumptions

05 · OBSERVABILITY
Attribute failures
Logs, replay, audit

06 · HUMAN BOUNDARY
When to stop, who owns
Automate execution, gate judgment

Low-hanging fruit: formal, scorable, cheap to verify · **Real systems problem**: reproducible, scalable, maintainable

> Low-hanging fruit proves direction; **the hard part is the whole machine**.

From the next token to the next theorem

`token` → `probability` → `candidate` → `verification` → **`theorem`**

Probability gives breadth

Verification gives correctness

People set the direction

> The next theorem still begins with the **next token**.